

DEPARTMENT OF TELECOMMUNICATIONS
ENGINEERING



EXTENDED CEP PROJECT REPORT
LAB NUMBER 12 OEL
(OEL OPEN ENDED LAB)

S.NO	NAME	CLASS	ROLL NO
01	MUHAMMAD MURSALEEN	FETC	TC-029

SUBMITTED TO
DR AAMIR ZEB

INDEX OF THE REPORT

S.no	Components of report	page no
01	Abstract	03
02	Introduction	03
03	Literature review	04
04	System model diagram	05
05	Source code	08-10
06	Output	11
07	Applications	12
08	Challenges	12
09	Conclusion	13

OPEN ENDED LAB(OEL)

REPORT

ABSTRACT

Flappy bird is a 2D game made with python accessing through pygame library. This project teaches us how to do and understands complex programming, how to show some thing on the screen, how to move something on screen and how to enable the program for detecting collisions and scores.

INTRODUCTION

It is an extended CEP in the form of OEL which is our practical no 12 Flappy bird is popular and easy to play game. Flappy bird is a GUI based program that is a 2D game .In which we control a small bird to avoid colliding on the pipes and move it thorough the pipes. This game may look simple but making score in this game is quite difficult as we have only one key to control it. And the way between the pipes are narrow, maintaining the bird at same level is an actual task.

LITERATURE REVIEW

INTRODUCTION

Flappy bird is a traditional and very famous game for beginner level GUI based programming. It is inspired by many other collision avoiding games. Such as chrome dino .It involves one button control.

GAME CONCEPT

- Game loops that runs continuously
- Adding physics concept to the game like gravity and delta time
- Including sprite animation to the game
- Detecting collisions
- Scrolling game background endlessly
- Sound on each movement
- High score saving

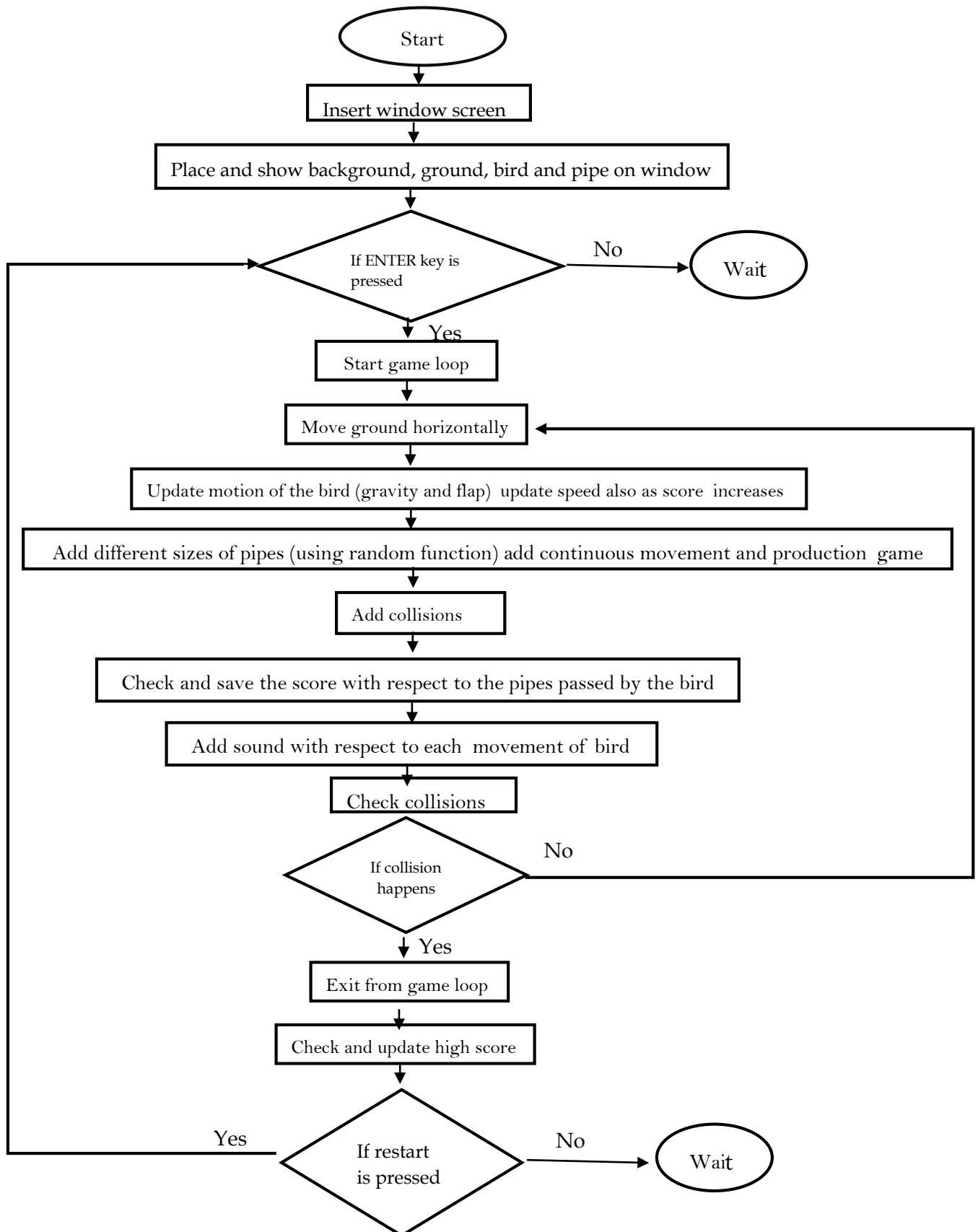
PREVIOUS GAME RELATED PROJECT

Those projects were also 2D having single key control applied gravity and velocity accordingly.

RELEVANCE TO MY PROJECT

Our project is also a 2D project having one key control and many concepts of physics are involved in this project

SYSTEM MODEL DIAGRAM



SOURCE CODE

Flappybird.py

```
1  import pygame as pg
2  import sys,time
3  from Bird import Bird
4  from pipe import Pipe
5
6  pg.init()      #Initialize all pygame modules
7  pg.mixer.init() # Initialize the mixer for sound
8
9  class Game:
10     def __init__(self):      # Self refers to the specific object being created
11         # Setting Window Configuration
12         self.width = 600
13         self.height = 600
14         self.scale_factor=1.5
15         self.win=pg.display.set_mode((self.width,self.height)) #Create and store the game window
16         self.clock=pg.time.Clock() # we create clock object
17         self.move_speed=250
18         self.start_monitoring=False
19         self.hit_sound_played = False
20         self.score=0
21         self.last_speed_increase_score = 0
22         self.font=pg.font.Font("assets/font.ttf",24)
23         self.score_text=self.font.render("Score: 0",True,(255,255,255))
24         self.score_text_rect=self.score_text.get_rect(center=(100,30))
25
26         self.restart_text=self.font.render("Restart",True,(0,0,0))
27         self.restart_text_rect=self.restart_text.get_rect(center=(300,600))
28         self.gameover_text = self.font.render("GAME OVER", True, (255, 0, 0))
29         self.gameover_text_rect = self.gameover_text.get_rect(center=(300, 300))
30
31     # Load Sound Effects
32     self.flap_sound = pg.mixer.Sound("assets/flap.wav")
33     self.score_sound = pg.mixer.Sound("assets/score.wav")
34     self.sound_hit=pg.mixer.Sound("assets/hit.wav")
35
36
37     # Load high score from file
38     try:
39         with open("highscore.txt", "r") as f:
40             self.high_score = int(f.read())
41     except FileNotFoundError:
42         self.high_score = 0 # If no file, start with 0
43
44     self.bird=Bird(self.scale_factor) # We pass scale factor to Bird object to scale it
45     self.is_enter_pressed=False
46     self.is_game_started=True
47     self.pipes=[] # We create pipes list in which current no pipes are present, its empty at beginning
48     self.pipe_generate_counter=71
49     self.setUpBgandGround()
50     self.gameLoop()
51
52
53     def gameLoop(self):
54         last_time=time.time()
55         while True: #we create infinite loop
56             # Calculating delta time #
57             new_time=time.time()
58             dt=new_time-last_time
```

```

59     last_time=new_time
60     for event in pg.event.get(): #pygame store all the events, we run for loop on events to get any event which we required
61         if event.type == pg.QUIT:
62             pg.quit() #we call pg.quit() method to close pygame only
63             sys.exit() #to close loop we use sys module and from this we call sys.exit() method
64         if event.type==pg.KEYDOWN and self.is_game_started:
65             if event.key==pg.K_RETURN: #Return = enter Key
66                 self.is_enter_pressed=True
67                 self.bird.update_on=True
68             if event.key==pg.K_SPACE and self.is_enter_pressed: #We only press space key when enter pressed
69                 self.bird.flap(dt)
70                 self.flap_sound.play()
71         if event.type==pg.MOUSEBUTTONDOWN:
72             if self.restart_text_rect.collidepoint(pg.mouse.get_pos()):
73                 self.restartGame()
74
75
76     self.updateEverything(dt) # Before drawing we have to update everything
77     self.checkcollisions()
78     self.checkScore()
79     self.drawEverything()
80     pg.display.update() #If we dont update,so nothing shows on window
81     self.clock.tick(60) # We lock game to run only at 60 fps
82
83     def restartGame(self):
84         self.score = 0
85         self.score_text = self.font.render("Score: 0", True, (0,0,0))
86         self.is_enter_pressed = False
87         self.is_game_started = True
88
89         self.bird.resetPosition()
90         self.pipes.clear()
91         self.pipe_generate_counter = 71
92         self.bird.update_on = False
93         # Reset collision sound flags
94         self.hit_pipe_played = False
95         self.hit_ground_played = False
96
97     def checkScore(self):
98         if len(self.pipes)>0:
99             if (self.bird.rect.left>self.pipes[0].rect_down.left and
100                 self.bird.rect.right<self.pipes[0].rect_down.right and not self.start_monitoring):
101                 self.start_monitoring=True
102             if self.bird.rect.left > self.pipes[0].rect_down.right and self.start_monitoring:
103                 self.start_monitoring=False
104                 self.score+=1
105                 self.score_text=self.font.render(f"Score: {self.score}",True,(255,255,255))
106                 self.score_sound.play() # Play score sound
107
108                 if self.score % 5 == 0 and self.score != self.last_speed_increase_score:
109                     self.move_speed *= 1.05
110                     self.last_speed_increase_score = self.score
111
112
113     def checkcollisions(self):
114         if len(self.pipes):
115             # Pipe collision
116             if (self.bird.rect.colliderect(self.pipes[0].rect_down) or

```

```

117         self.bird.rect.colliderect(self.pipes[0].rect_up)):
118         if not getattr(self, "hit_pipe_played", False):
119             self.sound_hit.play()
120             self.hit_pipe_played = True
121         self.is_enter_pressed = False
122         self.bird.update_on = True # gravity still affects bird
123         self.bird.flap_speed = 0
124
125     # Ground collision
126     if self.bird.rect.bottom >= 500:
127         self.bird.rect.bottom = 500
128         self.bird.y_velocity = 0
129         self.bird.update_on = False
130         self.is_enter_pressed = False
131         self.is_game_started = False
132         # Play ground sound only if bird did NOT hit pipe first
133         if not getattr(self, "hit_pipe_played", False) and not getattr(self, "hit_ground_played", False):
134             self.sound_hit.play()
135             self.hit_ground_played = True
136
137     # Update high score if game over
138     if not self.is_game_started and self.score > self.high_score:
139         self.high_score = self.score
140         with open("highscore.txt", "w") as f:
141             f.write(str(self.high_score))
142
143
144     def updateEverything(self,dt):
145         if self.is_enter_pressed: #if enter pressed then all the code below runs otherwise not
146
147             # moving the ground
148             self.ground1_rect.x-=int(self.move_speed*dt)
149             self.ground2_rect.x-=int(self.move_speed*dt)
150
151             if self.ground1_rect.right<0:
152                 self.ground1_rect.x=self.ground2_rect.right
153             if self.ground2_rect.right<0:
154                 self.ground2_rect.x=self.ground1_rect.right
155
156             # generating pipes
157             if self.pipe_generate_counter>71:
158                 self.pipes.append(Pipe(self.scale_factor,self.move_speed)) #So we append 1 pipe in pipes list
159                 self.pipe_generate_counter=0 #we reset pipe generate counter to 0
160             self.pipe_generate_counter+=1 # we increarse by 1
161
162             #moving pipes
163             for pipe in self.pipes:
164                 pipe.update(dt)
165
166             #removing pipes if out of screen
167             if len(self.pipes)!=0: # If pipe list is not empty
168                 if self.pipes[0].rect_up.right<0: #if right side corner of 0th index pipe becomes less than 0
169                     self.pipes.pop(0) #so delete the 0th index pipe
170
171
172             #moving the Bird
173             self.bird.update(dt) # Update bird position and animation
174

```

```

175
176     def drawEverything(self):
177
178         self.win.blit(self.bg_img,(0,-320))
179         for pipe in self.pipes:
180             pipe.drawPipe(self.win)
181         self.win.blit(self.ground1_img,self.ground1_rect)
182         self.win.blit(self.ground2_img,self.ground2_rect)
183         self.win.blit(self.bird_image,self.bird_rect)
184         self.win.blit(self.score_text,self.score_text_rect)
185         high_score_text = self.font.render("High Score: {self.high_score}", True, (255, 255, 255))
186         high_score_rect = high_score_text.get_rect(center=(450, 30))
187         self.win.blit(high_score_text, high_score_rect)
188
189         if not self.is_game_started:
190             self.win.blit(self.restart_text,self.restart_text_rect)
191         if not self.is_game_started:
192             self.win.blit(self.gameover_text, self.gameover_text_rect)
193
194
195     def setUpBgandGround(self):
196         # Loading images for bg and ground
197         self.bg_img=pg.transform.scale_by(pg.image.load("assets/bg.png").convert(),self.scale_factor) #pygame call load function from image module
198         self.ground1_img=pg.transform.scale_by(pg.image.load("assets/ground.png").convert(),self.scale_factor)
199         self.ground2_img=pg.transform.scale_by(pg.image.load("assets/ground.png").convert(),self.scale_factor)
200
201         self.ground1_rect=self.ground1_img.get_rect() #we consider rectangle around ground1 img to get its parameters(x,y position,height etc)
202         self.ground2_rect=self.ground2_img.get_rect()
203
204
205         #x-coordinates of ground
206         self.ground1_rect.x=0
207         self.ground2_rect.x=self.ground1_rect.right
208
209         #y-coordinates of ground
210         self.ground2_rect.y=500 # y coordinate remains same bcuz only ground move in x axis
211         self.ground1_rect.y=500
212     game=Game()

```

Bird.py

```
1 import pygame as pg
2 class Bird(pg.sprite.Sprite): #we inherit Sprite class from sprite module so that our bird adopt all sprite features#
3     def __init__(self,scale_factor):
4         super(Bird,self).__init__() #we create super class and initialize it
5         self.img_list=[pg.transform.scale_by(pg.image.load("assets/birdup.png").convert_alpha(),scale_factor),
6                         pg.transform.scale_by(pg.image.load("assets/birddown.png").convert_alpha(),scale_factor)]
7         self.image_index=0
8         self.image=self.img_list[self.image_index] # we initially set 0th index image
9         self.rect=self.image.get_rect(center=(100,100)) # we consider rectangle around 0th index image and pass tuple( position of placing bird)
10        self.y_velocity=0
11        self.gravity=10
12        self.flap_speed=250
13        self.anim_counter=0
14        self.update_on=False
15
16    def update(self,dt):
17        if self.update_on:
18            self.playAnimation() #before apply gravity we have to update animation
19            self.applyGravity(dt)
20
21        if self.rect.y<=0: #if we want to move above from y=0
22            self.rect.y=0 # we stuck at y=0
23            self.flap_speed=0 #and consider flap speed=0 so bird stuck at y=0
24        elif self.rect.y>=0 and self.flap_speed==0:
25            self.flap_speed=250
26
27    def applyGravity(self,dt):
28        self.y_velocity+=self.gravity*dt #y coordinate velocity increases with gravity
29        self.rect.y+=self.y_velocity #the y position of bird increases with velocity along y axis
30
31    def flap(self,dt):
32        self.y_velocity-=self.flap_speed*dt
33
34    def playAnimation(self):
35        if self.anim_counter==5:
36            self.image=self.img_list[self.image_index]
37            if self.image_index==0: self.image_index=1
38            else: self.image_index=0
39            self.anim_counter=0 # we reset anim counter to 0
40
41        self.anim_counter+=1
42
43    def resetPosition(self):
44        self.rect.center=(100,100)
45        self.y_velocity=0
46        self.anim_counter=0
```

Pipe.py

```
1 import pygame as pg
2 import random,import,randint
3
4 class Pipe:
5     def __init__(self,scale_factor,mov_speed):
6         self.img_up=pg.transform.scale_by(pg.image.load("assets/pipeup.png").convert_alpha(),scale_factor)
7         self.img_down=pg.transform.scale_by(pg.image.load("assets/pipedown.png").convert_alpha(),scale_factor)
8         self.rect_up=self.img_up.get_rect() #we consider rect around pipes
9         self.rect_down=self.img_down.get_rect()
10        self.pipe_distance=150
11        self.rect_up.y=randint(250,520) # Set a random vertical position for the upper pipe between 250 and 520 means the pipes generating was act
12
13        self.rect_up.x=600 # The x position of pipe is same as window width so that pip starts from the right edge of the screen
14        self.rect_down.y=self.rect_up.y-self.pipe_distance-self.rect_up.height # to place pipedown image we use this formula
15        self.rect_down.x=600
16        self.move_speed=mov_speed
17
18    def drawPipe(self,win):
19        win.blit(self.img_up,self.rect_up)
20        win.blit(self.img_down,self.rect_down)
21
22    def update(self,dt):
23        self.rect_up.x-=int(self.move_speed*dt) # The x position of both pipes decreases with some speed
24        self.rect_down.x-=int(self.move_speed*dt)
```

OUTPUT



APPLICATIONS

This project teaches us to polish our programming skills by using python and pygame. Focusing on basic structure and logical thinking of a game and practicing different methods .It will act like a step for future programming. It help us to introduce animations and movement in our program.Now a little more features has been added to it which are sound high score saving mechanism.

CHALLENGES

Some challenges of this program are as follows:

- Insert the image and show them on the window screen
- Use of sprite class
- Adding animations to the bird so that it move (gravity and flap)
- Allow the ground to move continuously and placing the pipes on it that moves
- Also to introduce collision
- Measuring the score with respect to the 1st pipe
- Add sound
- Adding high score
- Adding game over at last
- Save high score

CONCLUSION

We have entitled our cep as lab no 12 which was basically Flappy bird is a game designed by using python. It combines concept of physics like delta time and gravity with pygame library that allows to introduce game related animations to a Program. It allowed us to do logical thinking and apply different concept sequence wise. By programming this game we got to know the basic structure of a game programming. We spent more than 70 hours in this project and achieved what our main goal was.